

Dokumentacja bazy danych – System biblioteczny (MySQL)

1. Założenia ogólne

- **Baza danych:** MySQL 8.0 (np. baza `library`).
- **Backend:** Java + Spring/JPA, **frontend:** React.
- **Role aplikacyjne (min. 2, zgodnie z wymaganiami):**
 - `READER` – zwykły czytelnik.
 - `ADMIN` – administrator/bibliotekarz.
- **Hasła:** przechowujemy tylko **hash** (np. BCrypt) w kolumnie `password_hash`.
- **Główne obszary danych:**
 - Użytkownicy + role + blokady kont.
 - Książki, autorzy, kategorie (hierarchiczne), egzemplarze.
 - Wypożyczenia, zwroty, prolongaty.
 - Rezerwacje (kolejka do książki bez dostępnych egzemplarzy).
 - Kary i obostrzenia.
 - Proste statystyki wypożyczeń (widoki).

2. Model pojęciowy – encje

Encje wynikają bezpośrednio z przypadków użycia czytelnika i administratora.

1. `app_user` – konto użytkownika (czytelnik / admin).
2. `book` – opis logicznej książki (tytuł, opis, kategoria, ISBN).
3. `author` – autor książki.
4. `book_author` – tabela łącząca książki z autorami (N:M).
5. `category` – kategorie książek (np. „Fantastyka”, „Nauka”) – obsługa hierarchii (`parent_id`).
6. `book_copy` – fizyczny egzemplarz książki (kod inwentarzowy).
7. `loan` – wypożyczenie egzemplarza dla użytkownika.
8. `reservation` – rezerwacja książki, gdy wszystkie egzemplarze są wypożyczone.
9. `penalty` – zapis kar/obostrzeń za przetrzymanie.

3. Typy ENUM (Mapowanie w aplikacji)

W MySQL używamy typu `VARCHAR` dla kolumn mapowanych jako `ENUM` w Java/JPA (`EnumType.STRING`):

- **UserRole:** `READER`, `ADMIN`
- **UserStatus:** `ACTIVE`, `BLOCKED`
- **BookCopyStatus:** `AVAILABLE`, `BORROWED`, `LOST`, `DAMAGED`, `WITHDRAWN`

- **LoanStatus:** ACTIVE, RETURNED, OVERDUE, LOST
 - **ReservationStatus:** ACTIVE, CANCELLED, FULFILLED, EXPIRED
 - **PenaltyStatus:** OPEN, PAID, CANCELLED
-

4. Relacje między encjami (opis ERD)

- app_user (1) – (N) loan
- app_user (1) – (N) reservation
- app_user (1) – (N) penalty
- book (1) – (N) book_copy
- author (N) – (N) book przez book_author
- category (1) – (N) book
- category (1) – (N) category (hierarchia)
- book_copy (1) – (N) loan
- book (1) – (N) reservation
- app_user (1) – (N) loan (jako twórca wypożyczenia - admin)

Najważniejsze zasady:

- **Wypożyczenie** dotyczy **konkretnego egzemplarza** (loan.book_copy_id).
 - **Rezerwacja** dotyczy **tytułu** (książki), nie konkretnego egzemplarza (reservation.book_id).
 - W aplikacji i na poziomie bazy (unikalne indeksy) pilnujemy, aby:
 - jeden egzemplarz nie był równocześnie w dwóch aktywnych wypożyczeniach,
 - użytkownik nie miał dwóch aktywnych rezerwacji tego samego tytułu.
-

5. Specyfikacja tabel

5.1. Tabela app_user

Opis: konta użytkowników (czytelnicy i administratorzy), logowanie, blokady.

```
CREATE TABLE app_user (  
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,  
  email       VARCHAR(255) NOT NULL UNIQUE,  
  password_hash VARCHAR(255) NOT NULL,  
  first_name  VARCHAR(255) NOT NULL,  
  last_name   VARCHAR(255) NOT NULL,  
  role        VARCHAR(50) NOT NULL, -- READER / ADMIN  
  status      VARCHAR(50) NOT NULL DEFAULT 'ACTIVE',  
  blocked_reason VARCHAR(255),  
  blocked_until DATETIME(6),  
  created_at   DATETIME(6) NOT NULL,  
  updated_at   DATETIME(6) NOT NULL  
);
```

5.2. Tabela category

Opis: kategorie książek (hierarchiczne).

```
CREATE TABLE category (  
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,  
  name        VARCHAR(255) NOT NULL UNIQUE,  
  parent_id   BIGINT,  
  created_at  DATETIME(6) NOT NULL,  
  CONSTRAINT fk_category_parent FOREIGN KEY (parent_id) REFERENCES category(id)  
);
```

5.3. Tabela **author**

Opis: autorzy książek.

```
CREATE TABLE author (  
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,  
  first_name  VARCHAR(255),  
  last_name   VARCHAR(255) NOT NULL,  
  created_at  DATETIME(6) NOT NULL  
);
```

5.4. Tabela **book**

Opis: logiczna pozycja w katalogu (nie egzemplarz).

```
CREATE TABLE book (  
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,  
  title       VARCHAR(255) NOT NULL,  
  description  VARCHAR(255),  
  publication_year SMALLINT,  
  isbn        VARCHAR(255) UNIQUE,  
  category_id BIGINT,  
  is_active   BOOLEAN NOT NULL DEFAULT TRUE,  
  created_at  DATETIME(6) NOT NULL,  
  updated_at  DATETIME(6) NOT NULL,  
  CONSTRAINT fk_book_category FOREIGN KEY (category_id) REFERENCES category(id)  
);
```

5.5. Tabela **book_author**

Opis: relacja N:M między książkami a autorami.

```
CREATE TABLE book_author (
  book_id      BIGINT NOT NULL,
  author_id    BIGINT NOT NULL,
  PRIMARY KEY (book_id, author_id),
  CONSTRAINT fk_ba_book FOREIGN KEY (book_id) REFERENCES book(id),
  CONSTRAINT fk_ba_author FOREIGN KEY (author_id) REFERENCES author(id)
);
```

5.6. Tabela **book_copy**

Opis: fizyczne egzemplarze książek.

```
CREATE TABLE book_copy (
  id            BIGINT AUTO_INCREMENT PRIMARY KEY,
  book_id       BIGINT NOT NULL,
  inventory_code VARCHAR(255) NOT NULL UNIQUE,
  status        VARCHAR(50) NOT NULL DEFAULT 'AVAILABLE',
  shelf_location VARCHAR(255),
  created_at    DATETIME(6) NOT NULL,
  updated_at    DATETIME(6) NOT NULL,
  CONSTRAINT fk_copy_book FOREIGN KEY (book_id) REFERENCES book(id)
);
```

5.7. Tabela **loan**

Opis: pojedyncze wypożyczenie egzemplarza książki przez użytkownika.

```
CREATE TABLE loan (
  id            BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_id       BIGINT NOT NULL,
  book_copy_id  BIGINT NOT NULL,
  loan_date     DATETIME(6) NOT NULL,
  due_date      DATETIME(6) NOT NULL,
  return_date   DATETIME(6),
  status        VARCHAR(50) NOT NULL,
  extensions_count SMALLINT NOT NULL DEFAULT 0,
  created_by    BIGINT,
  created_at    DATETIME(6) NOT NULL,
  CONSTRAINT fk_loan_user FOREIGN KEY (user_id) REFERENCES app_user(id),
  CONSTRAINT fk_loan_copy FOREIGN KEY (book_copy_id) REFERENCES book_copy(id),
  CONSTRAINT fk_loan_admin FOREIGN KEY (created_by) REFERENCES app_user(id)
);
```

5.8. Tabela **reservation**

Opis: rezerwacje książki, gdy brak wolnych egzemplarzy.

```
CREATE TABLE reservation (  
  id                BIGINT AUTO_INCREMENT PRIMARY KEY,  
  user_id           BIGINT NOT NULL,  
  book_id           BIGINT NOT NULL,  
  status            VARCHAR(50) NOT NULL DEFAULT 'ACTIVE',  
  created_at        DATETIME(6) NOT NULL,  
  cancelled_at      DATETIME(6),  
  fulfilled_at      DATETIME(6),  
  expires_at        DATETIME(6),  
  CONSTRAINT fk_res_user FOREIGN KEY (user_id) REFERENCES app_user(id),  
  CONSTRAINT fk_res_book FOREIGN KEY (book_id) REFERENCES book(id)  
);
```

5.9. Tabela `penalty`

Opis: kary za przetrzymanie książek.

```
CREATE TABLE penalty (  
  id                BIGINT AUTO_INCREMENT PRIMARY KEY,  
  user_id           BIGINT NOT NULL,  
  loan_id           BIGINT,  
  amount            DECIMAL(10,2) NOT NULL DEFAULT 0.00,  
  reason            VARCHAR(255) NOT NULL,  
  status            VARCHAR(50) NOT NULL DEFAULT 'OPEN',  
  created_at        DATETIME(6) NOT NULL,  
  resolved_at       DATETIME(6),  
  CONSTRAINT fk_penalty_user FOREIGN KEY (user_id) REFERENCES app_user(id),  
  CONSTRAINT fk_penalty_loan FOREIGN KEY (loan_id) REFERENCES loan(id)  
);
```

6. Indeksy i kluczowe ograniczenia

Oprócz PK/FK warto dodać:

```
-- Szybsze wyszukiwanie po tytule  
CREATE INDEX idx_book_title ON book (title);  
  
-- Szybsze filtrowanie po dacie wypożyczenia  
CREATE INDEX idx_loan_date ON loan (loan_date);
```

7. Widoki statystyczne (Logika aplikacji)

W systemie MySQL widoki mogą być wykorzystywane do generowania raportów dostępnych w panelu administratora:

1. **Popularność książek** (najczęściej wypożyczane).
 2. **Dzienny raport wypożyczeń.**
-

8. Mapowanie przypadków użycia → model danych

8.1. Czytelnik

- **Profil i logowanie** → `app_user`.
- **Katalog książek** → `book`, `author`, `category`.
- **Wypożyczenia i Rezerwacje** → `loan`, `reservation`.
- **Kary** → `penalty`.

8.2. Administrator

- **Zarządzanie zasobami** → `book`, `book_copy`, `author`, `category`.
- **Zarządzanie wypożyczeniami** → `loan` (rejestracja zwrotów, przedłużeń).
- **Zarządzanie użytkownikami** → `app_user`, `penalty`.
- **Statystyki** → Dashboard na podstawie danych z `loan` i `book`.